# MODULES IN PYTHON

# INTRODUCTION

A module is a single Python file (.py) that contains Python code: functions, classes, variables, and even runnable code.

**Purpose:**
To **organize code**, increase reusability, improve readability, and avoid duplication.

**Why Use Modules?**

- Code reusability
- Better organization (large projects become manageable)
- Separation of concerns
- Easier testing and maintenance

# CREATING A MODULES

**Creating a module**

**Step 1: Create a file named mymodule.py**

**Step 2 : Define your code**

```
# mymodule.py

def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b

pi = 3.14159
```

# IMPORTING A MODULE

**Using import statement**

**Method - 1**

Syntax: **import module**

Example : **import mymodule**

Note : Here Python imports only the module, NOT all its names into your current namespace.

So, everything inside mymodule is loaded, but not directly accessible unless you use the module prefix:

Example: print(mymodule.add(10,5))

print(mymodule.greet('QUEST'))

# IMPORTING A MODULE

**Method - 2**

Syntax: **from module_name import name1, name2, name3, ...**

Example : **from math import sqrt, pi**

Note :

- Here only sqrt and pi are imported
- You can use them directly
- Does NOT import the entire module

Example : print(sqrt(16))

print(pi)

# IMPORTING A MODULE

**Method - 3**

Syntax: **from module_name import ***

Example : **from math import ***

Note :

- This imports **all public functions, variables & classes** of the module into your namespace directly.
- No need for math.sqrt, just sqrt(16)
- Can cause **name conflicts**
- **Hard to debug**
- **Not recommended in real project**
- **Only safe for small classroom examples**

# IMPORTING A MODULE

**Method - 4**

Syntax:

**from module_name import  name1 as alias1, name 2 as alias2….**

Example : **from math import sqrt as s, pi as p**

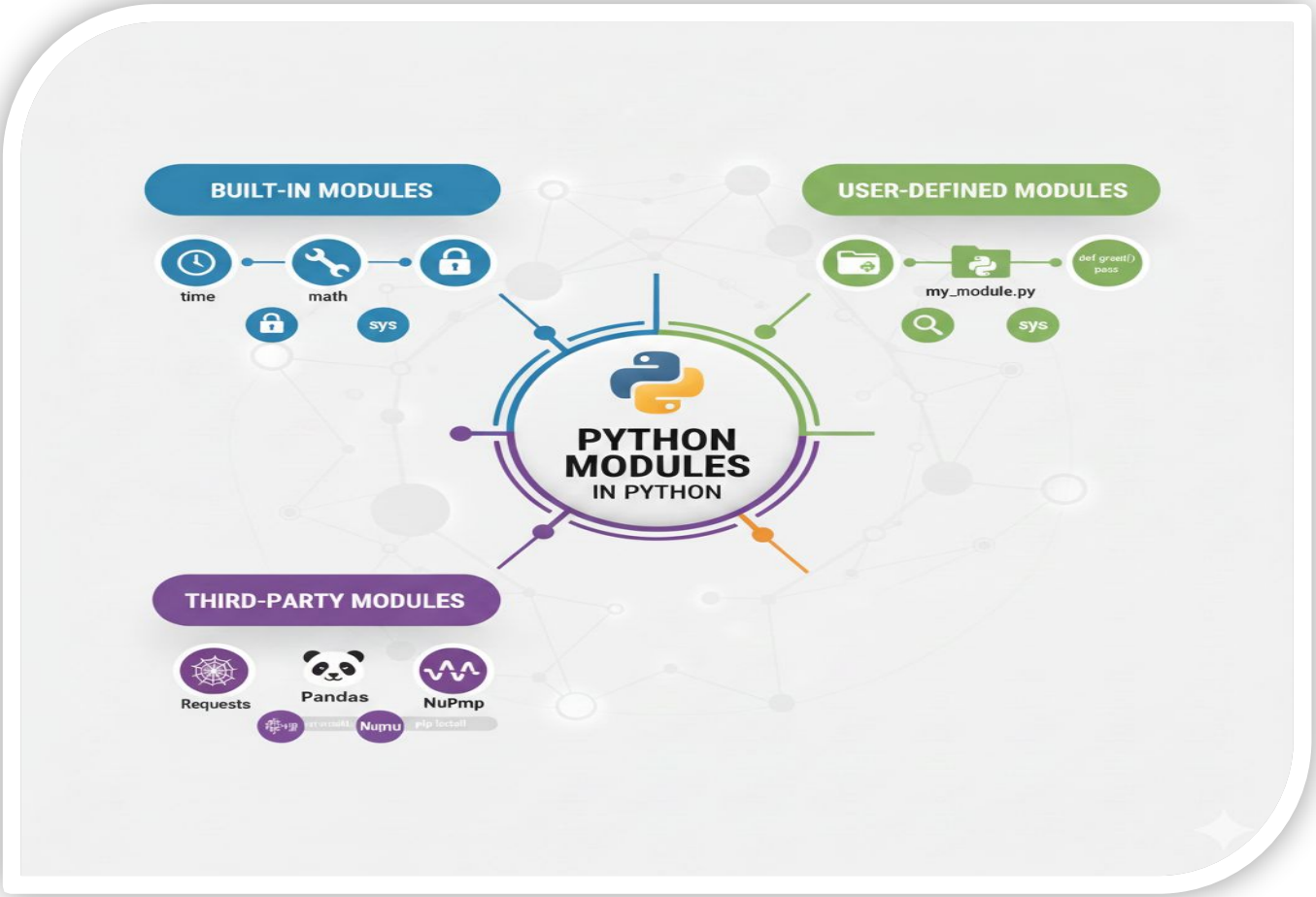
**Advantages**

- Avoid Name Conflicts
- Shorter and Cleaner Code
- Improves Readability & Meaning
- Avoid Overwriting Built-ins
- Aliases only rename in your program (not globally)

**Notes / cautions:**

- Use meaningful aliases
- Don’t overuse
- Follow community standards
- Alias only when necessary

# TYPES OF MODULES IN PYTHON



# Math Module

| Method | Description | Example |
|---|---|---|
| math.sqrt(x) | Square root of x | math.sqrt(25) |
| math.pow(x, y) | x raised to power y | math.pow(2, 3) |
| math.factorial(x) | Factorial of x | math.factorial(5) |
| math.ceil(x) | Round UP to nearest integer | math.ceil(4.1) |
| math.floor(x) | Round DOWN to nearest integer | math.floor(4.9) |
| math.trunc(x) | Remove decimal part | math.trunc(5.8) |
| math.fabs(x) | Absolute value (float) | math.fabs(-7.5) |

provides a wide range of **mathematical functions and constants** for performing mathematical operations

It contains:

- Mathematical **functions** (e.g., sqrt, sin, cos, log, ceil, floor)
- Mathematical **constants** (e.g., pi, e, tau, inf, nan)

# datetime module

- Used to work with **dates**, **times**, and **time intervals**.
- It allows you to create, manipulate, format, and perform arithmetic on **dates**, **times**, **timestamps**, and **durations**.

**1. Classes for working with date & time**

- date → works with only **year, month, day**
- time → works with only **hour, minute, second**
- datetime → combines **date & time**
- timedelta → represents **difference between two dates or times**

**2. Functions for current date & time**

- datetime.now() → current date & time
- date.today() → current date

**3. Formatting / Parsing functions**

- strftime() → convert datetime to string
- strptime() → convert string to datetime

**4. Time zone support**

- timezone class → create timezone-aware datetime values

# Strftime Formatting codes

| Code | Meaning | Example Output |
|---|---|---|
| **%Y** | Year (4 digits) | 2025 |
| **%y** | Year (2 digits) | 25 |
| **%m** | Month (01–12) | 03 |
| **%B** | Full month name | March |
| **%b** | Abbreviated month name | Mar |
| **%d** | Day of month (01–31) | 09 |
| **%A** | Full weekday name | Sunday |
| **%a** | Abbreviated weekday name | Sun |
| **%H** | Hour (00–23) | 17 |
| **%I** | Hour (01–12) | 05 |
| **%p** | AM/PM | PM |
| **%M** | Minute (00–59) | 42 |

# Strftime Formatting codes

| Code | Meaning | Example Output |
|---|---|---|
| **%S** | Seconds (00–59) | 33 |
| **%f** | Microseconds | 548321 |
| **%z** | UTC offset | +0530 |
| **%Z** | Timezone name | IST |
| **%j** | Day of year (001–366) | 123 |
| **%U** | Week number (Sunday first) | 22 |
| **%W** | Week number (Monday first) | 21 |
| **%c** | Full date & time (locale based) | Sun Mar 9 17:42:00 2025 |
| **%x** | Locale date | 03/09/25 |
| **%X** | Locale time | 17:42:00 |

# Random Module

- Used to generate **pseudo-random numbers**
- Example choosing random items, shuffling sequences, generating random integers, floats, and making random selections.

| Method | Description |
|---|---|
| **random()** | Returns a random float between **0.0 and 1.0**. |
| **randint(a, b)** | Returns a random integer between **a and b (both included)**. |
| **randrange(start, stop, step)** | Returns a random number from the given range (stop excluded). |
| **uniform(a, b)** | Returns a random **float** between **a and b**. |
| **choice(sequence)** | Returns a **random element** from a non-empty sequence. |
| **choices(sequence, k=n)** | Returns a list of **k randomly selected elements** (with replacement). |
| **shuffle(list)** | Randomly reorders the elements of a list **in place**. |
| **sample(population, k)** | Returns **k unique random elements** from a sequence (without replacement). |
| **seed(n)** | Sets the seed value for reproducing the same random results. |
| **betavariate(alpha, beta)** | Returns a random float from the **beta distribution**. |
| **expovariate(lambd)** | Returns a random float from an **exponential distribution**. |
| **gauss(mu, sigma)** | Returns a random float from a **Gaussian (normal) distribution**. |